

M08 / ASSESSMENT

M08 Module Revision Test

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

Rules 3

T01 - DE08-01 | Ingestion architecture: source -> landing -> staging 3

T02 - DE08-02 | Defensive CSV ingestion 3

T03 - DE08-03 | JSON and nested-record ingestion 3

T04 - DE08-04 | Parquet ingestion and metadata inspection 4

T05 - DE08-05 | REST API contracts, status codes and timeouts 4

T06 - DE08-06 | API authentication, pagination and rate limits 4

T07 - DE08-07 | Retries, backoff, jitter and transient failures 5

T08 - DE08-08 | Database extraction and safe query boundaries 5

T09 - DE08-09 | Historical batch landing and raw metadata 6

T10 - DE08-10 | Incremental loads and high-water marks 6

T11 - DE08-11 | CDC concepts and change-event semantics 6

T12 - DE08-12 | Schema drift, compatibility, time zones and quarantine 7

T13 - DE08-13 | Checkpoint/state management and replay safety 7

T14 - DE08-14 | Idempotent landing and content identity 7

T15 - DE08-15 | Late-arriving data, event time and processing time 8

T16 - DE08-16 | Ingestion observability: counts, latency and run metadata 8

T17 - DE08-17 | Ingestion integration, reconciliation and backfill proof 8

Read this first

MODULE REVISION TEST

Save your answers before opening the protected answer guide. No forced timer. First prove the competency untimed; later you may record interview-style time if useful. This test does not replace a protected Gate.

Coverage: 17 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

Rules

Work with learner book, review book, answer guide and AI solving closed. Save one answer/evidence item for every lesson. If a task needs a PC, write your prediction first, then run it. Mark PASS only when the result and your explanation agree.

T01 - DE08-01 | Ingestion architecture: source -> landing -> staging

A vendor sends the same filename orders.csv every day. Design a landing convention that proves which bytes belonged to Monday and Tuesday even though the source filename is identical. Explain why renaming the newest file in place is weaker than a run-scoped immutable landing copy.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T02 - DE08-02 | Defensive CSV ingestion

Create a new CSV with a quoted comma in a product description, one duplicate order_id, and one leading-zero customer ID. Predict what a naïve `line.split(',')` parser would do. Then use the CSV parser correctly and prove the identifier's leading zeros survive.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T03 - DE08-03 | JSON and nested-record ingestion

Add an order J3003 with an empty items array and a customer object missing name. Decide what belongs in an order-header table, what belongs in item staging, and what (if anything) should be quarantined. Do not solve by forcing a fake item row unless the destination contract explicitly requires one.

Work the grain by hand before coding For J3001, order grain gives one header with two items. Item grain gives (J3001,1) and (J3001,2). J3002 gives one item row. Therefore the fixture should produce three item rows total. Compute gross two ways: from nested source arrays and from flattened rows. J3001 is $2 \times 120 + 1 \times 300 = 540$; J3002 is $4 \times 75.5 = 302$; combined item gross is 842. If flattened total differs, the bug is in expansion or type conversion.

A subtle trap appears when parent fields are repeated. Suppose the order header contains `shipping_fee=20`. Repeating 20 on each J3001 item is fine for context, but summing it at item

grain would charge 40. The same lesson from SQL cardinality now appears during JSON normalization: repeating a descriptor is not permission to aggregate it.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T04 - DE08-04 | Parquet ingestion and metadata inspection

Write a second Parquet file with a deliberately different row-group size. Compare metadata and explain what changed physically while the logical table remains the same.

Read the metadata like evidence After writing the tiny file with `row_group_size=2`, four source rows should normally produce two row groups. Record what your actual PyArrow version reports rather than copying that expectation blindly. Inspect

schema types: a customer identifier loaded incorrectly as integer may lose 0001 even though Parquet itself is perfectly capable of storing strings. The file format cannot rescue a wrong DataFrame dtype chosen before writing.

Projection proof is simple: request only `order_id`, `updated_at`, `qty`. The resulting table should contain those columns and the same four logical rows. This does not prove every engine performed predicate pushdown; it proves column projection and logical parity.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T05 - DE08-05 | REST API contracts, status codes and timeouts

Imagine an endpoint returns status 200 with body `{"status":"error","items":[]}`. Decide whether HTTP success alone permits checkpoint advancement. Write the additional application-level contract you would need.

Build a response decision table

Before coding a generic handler, write what the client should do. 200 + expected contract -> continue. 200 + wrong body shape -> contract failure, do not checkpoint as clean success. 401 -> authentication/configuration failure, stop and repair credentials. 404 -> endpoint/path contract problem. 429 -> rate-limit handling belongs to DE08-06/07. 500 -> candidate transient failure under bounded retry.

A second axis is transport failure: DNS/connection/timeout may produce no HTTP response at all. Your error evidence should distinguish "server replied 500" from "no response before timeout" because diagnosis and retry policy differ.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T06 - DE08-06 | API authentication, pagination and rate limits

Set page size 4. Predict page count and sizes before execution. Then change only the pagination contract in a small mock response so the next cursor is a string token rather than a page number. Explain why production code should isolate pagination mechanics instead of scattering `page += 1` through transformation logic.

Prove completeness, not just page count With ten records and `page_size=3`, the expected IDs are A4001 through A4010. Four successful page payloads are necessary but not sufficient: page 2 could accidentally repeat A4001-A4003. Therefore validate both count and identity. For this fixture, `len(rows)=10`, `len(set(order_id))=10`, first/last IDs and sorted sequence are easy independent controls.

When the one-time 429 occurs on page 2, the trace should show page 1 success, page 2 rate limit, page 2 success, then remaining pages. Page 1 should not be appended twice. The retry belongs to the current page request, not to restarting the entire extraction unless your design deliberately makes whole-run

replay idempotent.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T07 - DE08-07 | Retries, backoff, jitter and transient failures

You now receive 503 three times and then 200. Maximum attempts is 3 total, not 3 retries after the first call. Predict whether success is reachable under that definition, then make the attempt-count wording explicit in code and logs.

Make attempt semantics unambiguous Teams often say "three retries" when one person means three total calls and another means one original plus three retries. The course code uses `max_attempts`: the total number of calls including the first. Log `attempt=1/4` style evidence so operations can interpret it.

For the one-time 500 test, the trace should contain 500 then 200. For 429, it should contain 429 then 200 while respecting Retry-After. For wrong token, the first 401 should raise without consuming all attempts. For an unreachable port, connection errors may consume the bounded attempt budget.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T08 - DE08-08 | Database extraction and safe query boundaries

The source gains a column `internal_notes`. Your extraction should remain unchanged because it names explicit fields. Prove it, then explain why schema drift in unused source columns may be non-breaking while drift in a consumed field may be breaking.

Prove the source stayed unchanged

Before extraction, record source row count and optionally file/database hash for the local fixture. After the read-only query, verify the same control. The point is not that SELECT usually mutates; it is to build the habit that source safety is observable.

The expected extraction after (08:15,D5002) is D5003 followed by D5004. D5003 shares the timestamp with D5002 but its ID is greater, so it qualifies. This is the same composite-ordering pattern later used for incremental APIs.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T09 - DE08-09 | Historical batch landing and raw metadata

The second file is resent with one corrected byte but the same name. Show how content hash reveals a new version. Decide whether to keep both landed copies and what lineage fields staging needs to identify

which one was used.

Historical ingestion needs a receipt Treat the manifest like a receiving log. It tells a future engineer what objects were acquired in one run before interpretation. For the two supplied files, record both original basenames, byte lengths and independent hashes. The manifest itself should have a run_id and acquisition time/owner/source where appropriate.

Do not rewrite the manifest entry after cleaning to contain the cleaned row count as if it described the raw file. You may add derived metadata in a separate field or downstream run record, but keep acquisition evidence distinguishable from transformation evidence.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T10 - DE08-10 | Incremental loads and high-water marks

Use a fresh fixture where three IDs share the same timestamp and the run intentionally stops after the second tied ID. Persist (timestamp, second_id) and prove the third ID is returned on resume. Then show what a timestamp-only watermark would do.

Walk the tied timestamp slowly The sequence contains A4007 at 09:00 and A4008 at 09:00. Suppose a run commits state after A4007 due to a controlled batch boundary. State (09:00,A4007) means the next query can safely include A4008 with timestamp = 09:00 AND id > A4007. State 09:00 alone cannot express that position.

The final checkpoint is (09:20,A4010) only after all ten records are durably accepted for the run. A clean rerun should produce zero eligible records. That zero is not "nothing happened"; it is evidence that the watermark excludes already processed rows as designed.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T11 - DE08-11 | CDC concepts and change-event semantics

Receive an update event before its create because files arrived out of order. Decide whether to quarantine, buffer, reconstruct from snapshot or allow upsert. Explain what evidence you need rather than inventing a universal rule.

Separate event log from current-state projection

The CDC fixture contains four events but final current state contains only A5001. That is not data loss: the event log preserves all four events, while the current-state projection applies create/update/delete semantics. These are two different grains: one row/event versus one row/current entity.

A5001 is created new, then updated paid. A5002 is created and later deleted. If you simply append every after object to a table, you get duplicate A5001 versions and may never represent the delete. If you only keep current state, you lose the audit/change history. Choose the target

intentionally.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T12 - DE08-12 | Schema drift, compatibility, time zones and quarantine

A v3 payload adds `discount_pct` but keeps all required v1 fields identical. Decide whether this is compatible for the v1 consumer and what metadata you would record so the ignored new field is not invisible to governance.

Compatibility belongs to the consumer contract

A producer can truthfully announce "schema v2" while your v1 consumer still cannot safely use it. Compatibility is therefore relative to a consumer and its expectations. The lesson's safest default is: land v2, classify as unsupported for v1 staging, quarantine with reason, and do not advance v1 state.

Time meaning is part of schema even when the JSON type is still "string". 09:00Z, 14:30+05:30 and 09:00 are all strings, but only the first two identify an instant without extra assumptions. Schema validation that checks only Python type misses semantic drift.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T13 - DE08-13 | Checkpoint/state management and replay safety

Create two independent ingestion jobs that accidentally share one checkpoint path. Explain the race/ownership problem and redesign state paths so each source/contract has a clear owner.

State is a commit record, not a scratch note Take a hash of the live checkpoint before replay. Run replay with no-commit state. Hash it again. Equality is a simple proof that replay did not alter live resume position. The replay may still create a new landing run; those are separate concerns.

Atomic replacement protects against a process reading a half-written JSON file, but it does not prevent two writers racing to replace the same path. Ownership/locking/orchestration is a higher-level concern. This

distinction prevents overclaiming what `os.replace` guarantees.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T14 - DE08-14 | Idempotent landing and content identity

Create two files with different byte hashes but the same order IDs and values because column order differs. Explain why byte-level deduplication would keep both while business-key staging can still avoid duplicate logical rows.

Use the right identity at the right layer Suppose two landing runs contain byte-identical `orders.csv`. Their SHA-256 values match, proving content

identity. Their run IDs differ, proving separate acquisition attempts. Staging keyed by order_id can still contain one logical row/order. All three facts can be true simultaneously.

Now change only CSV column order. Byte hashes differ, but parsed business rows may be logically identical. A content-addressed raw store would see two byte objects; a business-key staging model can recognize the same entities. This is why one universal "dedupe key" is a design smell.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T15 - DE08-15 | Late-arriving data, event time and processing time

Change tolerance to 15 minutes. Predict which classifications change before running. Explain why a watermark/tolerance change can alter operational state and may require deliberate reprocessing rather than silently applying a new rule midstream.

Calculate lateness explicitly For L2, event_time is 09:10Z and arrival is 10:25Z: lag = 75 minutes, therefore late under a 60-minute tolerance. L1 has 5 minutes. L3 uses +05:30; convert both event and arrival to UTC before subtraction. This is exactly why offset-aware timestamps matter.

Sort the same events two ways. Event-time order answers business chronology. Arrival-time order answers what the pipeline saw and when. If those orders differ, that is expected in distributed ingestion, not proof one timestamp is wrong.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T16 - DE08-16 | Ingestion observability: counts, latency and run metadata

A run acquires 10 records, stages 9 and has 0 quarantine. Your status says SUCCESS. Use the count equation to detect silent loss and redefine the run status/alert condition.

Turn metrics into controls A useful run record can answer: Did acquisition complete? How many source records were seen? How many were accepted or rejected? How many retries happened? What checkpoint existed before and after? Which raw content hashes belong to this run? How long did it take?

For the simple record classification contract, assert source=accepted+quarantined. A run with source=10, accepted=9, quarantine=0 must not be green. The missing row may be a software bug even when every individual staged row looks valid.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T17 - DE08-17 | Ingestion integration, reconciliation and backfill proof

The package includes a different assessment fixture with IDs/timestamps changed. Complete it without reopening the first solution. Mastery means you can reconstruct the reasoning - source contract, identity, state, validation and replay - when the values change.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY